Merge-Based vs. Hybrid-Chunk SpMV:

A CPU-Centric Parallel Performance Analysis

Parallel and Distributed Computing

Base Paper: Merrill & Garland, "Merge-based Parallel Sparse Matrix-Vector Multiplication", SC'16

Muhammad Ibrahim Malik(23i-0694)

Syed Muhammad Faateh Haneef(23i-0688)

CS-E



1. Introduction

Sparse Matrix-Vector Multiplication (SpMV) is one of the most widely used kernels in scientific computing, appearing in iterative linear solvers, eigenvalue computations, graph analytics, and machine learning. Its performance is critical because applications invoke it repeatedly inside convergence loops, in many solvers, SpMV alone accounts for 80-90% of total wall-clock time.

Despite its conceptual simplicity, SpMV is difficult to parallelise efficiently. The fundamental obstacle is load imbalance: real-world sparse matrices have highly unequal numbers of nonzeros per row. Assigning one thread per row, the natural parallelisation strategy, leaves most threads idle while a few process extremely long rows, wasting the available parallel hardware.

This project investigates two algorithmic approaches to this problem. The first is the Merge-based CsrMV of Merrill and Garland (SC'16), which solves load imbalance by reframing the parallel decomposition as a merge-path problem. The second is our proposed Hybrid Chunking method, which extends the merge-path idea by adaptively applying it only where it is needed: for heavy (long) rows, while using a simpler row-based strategy for light (short) rows to eliminate unnecessary binary search overhead.

We evaluate both methods on seven real-world sparse matrices from the SuiteSparse collection, spanning two structural categories: structured (low coefficient of variation) matrices from scientific simulations, and skewed (high coefficient of variation) matrices from social networks and communication graphs. All experiments are run on a commodity multicore CPU using OpenMP, and we relate results to the hardware memory bandwidth ceiling using an empirical Roofline model.

2. Base Paper and Problem Context

2.1 The Base Paper

Merrill, D. and Garland, M., "Merge-based Parallel Sparse Matrix-Vector Multiplication," SC'16, IEEE Press, 2016.

The paper identifies a fundamental weakness in all contemporary CsrMV implementations: they decompose work at the row boundary, making performance strongly dependent on row-length distribution. For matrices with coefficient of variation (CV) above 2, both Intel MKL and NVIDIA cuSPARSE exhibit order-of-magnitude performance degradation.

2.2 The Core Insight

Merrill and Garland reframe the SpMV computation as a merge of two sorted lists: the CSR row-offset array (which encodes where each row ends) and the natural numbers (which index each nonzero). Every step on this merged path is exactly one unit of work, either finishing a row (moving right) or computing a multiply-add (moving down). The total path length is always rows + nonzeros, and slicing it into equal segments gives every thread the same number of steps regardless of matrix structure.

Each thread finds its starting position on the path using a binary search on the row-offset array, an O(log n) operation. A lightweight sequential fix-up pass then combines partial row sums for rows that span thread boundaries.

2.3 Key Results From the Paper

On dual-socket Intel Xeon E5-2690 (40 threads), merge-based achieves 21-23 GFLOPs on all three test matrices regardless of CV, compared to MKL dropping from 17.9 to 11.8 GFLOPs. On GPU, merge-based achieves up to 198x speedup over cuSPARSE on ASIC_320k (CV = 61.4). The paper also notes that on small GPU matrices, merge-based can be 50% slower than cuSPARSE due to binary search and fix-up overhead.

2.4 The Gap We Identified

The paper's CPU evaluation deliberately avoids architecture-specific optimisations to isolate the load-balance effect. It does not study: (1) whether binary search overhead on rows that are trivially light is a measurable cost on CPU, (2) how cache efficiency compares between methods as a function of matrix structure, and (3) whether a per-row adaptive strategy that bypasses binary search for light rows can outperform pure merge-path on skewed matrices. This project fills that gap.

3. Project Scope and Objectives

3.1 Scope

This project implements, measures, and critically analyses two parallel SpMV algorithms on CPU:

- Merge-based CsrMV: Algorithm 2 from Merrill & Garland SC'16, using OpenMP.
- Hybrid Chunking CsrMV: our proposed extension that classifies rows by length and applies merge-path only to heavy rows, using row-based parallelism for light rows.

We evaluate both on seven SuiteSparse matrices across two structural categories, sweep thread counts from 1 to the hardware maximum, and relate all results to the hardware memory bandwidth ceiling via an empirical Roofline measurement.

3.2 Research Questions

- Q1: Does merge-path's binary search overhead constitute a measurable performance penalty on CPU for skewed matrices where most rows are trivially short?
- Q2: Can Hybrid Chunking reduce this overhead while preserving load balance on heavy rows, leading to net improvement on skewed matrices?
- Q3: On structured matrices with uniform row lengths, does Hybrid Chunking's classification overhead become a net cost?
- Q4: Are both methods memory-bandwidth-bound (as expected for SpMV), and how close do they operate to the hardware ceiling?

3.3 Novelty Statement

Existing hybrid work (Chen et al., 2022) converts the matrix into a different storage format (HYB5 = SELL-C-σ + CSR5), requiring preprocessing. CAMLB-SpMV (Guo et al., ICPP 2024) measures cache-line-level imbalance on CPU but does not implement per-row adaptive decomposition. Our approach applies the decomposition switch at runtime inside the original CSR format, no format conversion, no preprocessing, no supplemental data structures.

4. Baseline Method

4.1 Sequential Reference

The sequential baseline computes SpMV using the standard two-loop algorithm (Algorithm 1 of Merrill & Garland): for each row, iterate over its nonzeros and accumulate the dot product. This is single-threaded, uses the CSR format directly, and serves as the correctness reference and performance denominator for all speedup calculations.

Measured sequential performance across our matrices ranges from 1.47 GFLOPs (soc-Slashdot) to 1.99 GFLOPs (hollywood-2009), consistent with the memory-bandwidth-bound nature of SpMV at ~8-12 bytes per nonzero.

4.2 Correctness Verification

Both parallel implementations are verified by comparing their output vector y against the sequential reference at single-precision tolerance. No mismatches were observed across any matrix or thread count in our experiments. The verification check is performed once before timing begins and reported explicitly in the benchmark output.

4.3 Measurement Protocol

All timing uses 100 iterations with the best-of-five warmup. We report wall-clock time measured with omp_get_wtime() for thread-parallel sections. Performance is reported as GFLOPs (2 * nnz / time), effective memory bandwidth in GB/s (computed as bytes transferred per SpMV / time), Roofline utilisation percentage (measured BW / peak stream BW), and speedup relative to sequential baseline.

5. Implementation Details

5.1 Language and Tools

- Language: C++17
- Parallelism: OpenMP 4.5 (#pragma omp parallel, schedule(static), schedule(dynamic,64))
- Compiler: GCC 13.3, flags: -O3 -fopenmp -ffloat-store -lm
- Matrix I/O: Matrix Market (.mtx) reader from Merrill & Garland open-source artifact
- Roofline: Empirical Stream-Triad bandwidth measurement (3-array, double precision, 10M elements, 5 iterations)

5.2 Algorithm A: Merge-based CsrMV

Implementation follows Algorithm 2 of the base paper exactly. The key steps are:

- Compute total merge path length: num_rows + num_nonzeros.
- Divide equally: items_per_thread = ceil(total / num_threads).
- Each thread performs a binary search (MergePathSearch) on the row_offsets array to find its 2D starting coordinate (row_idx, nz_idx) at its assigned diagonal.
- Each thread executes the sequential SpMV loop over its share of the path, accumulating partial sums across row boundaries.
- Sequential fix-up pass: for each thread boundary, add the carry-out partial sum to the corresponding row of y.

The binary search runs in O(log n_rows) time per thread. The fix-up pass is O(num_threads) and sequential. The main computation is O(nnz / num_threads) per thread.

5.3 Algorithm B: Hybrid Chunking CsrMV (Our Approach)

The hybrid method separates the matrix into two classes using a threshold computed as the 90th percentile of row lengths:

- Inspector pass (sequential): scan row_offsets once to classify every row as light (length <= threshold) or heavy (length > threshold). Collect heavy row indices into a compact array. O(n_rows).
- Pass 1: Light rows: schedule(dynamic, 64) OpenMP loop over all rows. If light, compute the row dot-product directly. If heavy, skip (y[i] = 0 as placeholder). No binary search.
- Pass 2: Heavy rows only: build a mini row-offset array for the heavy subset. Apply merge-path decomposition only over this reduced problem. Binary search cost is O(log n_heavy) which is much smaller than O(log n_rows) for skewed matrices.
- Fix-up: combine partial sums for heavy rows split across thread boundaries, exactly as in pure merge-path.

The key design insight is that for a matrix like soc-Epinions1, where 99.9% of rows have <= 10 nonzeros but 4 rows have >1000, the inspector identifies 8 heavy chunks from 75,884 light rows. The binary search is performed only over these 8 chunks, not over 75,888 rows. This is the source of the 1.34x speedup over pure merge-path at 1 thread.

5.4 Threshold Selection

The threshold is computed as the 90th percentile of the row-length distribution for each matrix. This is a one-time O(n log n) sort of the row lengths during the inspector phase. For the structured matrices (ecology2, apache2, G3_circuit), the threshold equals or exceeds the maximum row length, meaning all rows are classified as light and the heavy-row merge-path pass is skipped entirely. This is why Hybrid Chunking degrades on structured matrices, it adds inspector overhead and two-pass scheduling with no benefit from the merge-path pass.

6. Experimental Setup

6.1 Hardware Profile

System: HP ProBook 440 G9 Notebook PC (malik@malik-HP-ProBook-440-14-inch-G9-Notebook-PC)

CPU: 12 logical threads (hyperthreaded multicore)

Measured Peak Memory Bandwidth: ~16.9–17.1 GB/s (Stream Triad, varies ±5% run-to-run)

L3 Cache: Shared last-level cache; SpMV working sets for most matrices exceed cache capacity

OS: Ubuntu Linux

6.2 Matrix Dataset Summary

Matrix	Rows	NNZ	Mean RowLength	Std Dev	CV	Skewness

ecology2	999,999	4,995,991	4.996	0.063	**0.013**	-15.76
apache2	715,176	4,817,870	6.737	0.448	**0.067**	-1.19
G3_circuit	1,585,478	7,660,826	4.832	0.642	**0.133**	-0.75
email-Enron	36,692	367,662	10.020	36.100	**3.603**	16.30
soc-Epinions1	75,888	508,837	6.705	26.015	**3.880**	20.36
soc-Slashdot	82,168	948,464	11.543	36.918	**3.198**	19.02
hollywood-2009	1,139,905	113,891,327	99.913	271.865	**2.721**	7.35

Table 1: Dataset summary. Blue = structured (low CV), Yellow = skewed (high CV). CV = coefficient of variation of row lengths = std_dev / mean. Higher CV indicates greater irregularity.

The structured matrices (ecology2, apache2, G3_circuit) are from finite-element and circuit simulation domains. Their row lengths are nearly identical, ecology2 has CV = 0.013 and max row length 5. The skewed matrices (email-Enron, soc-Epinions1, soc-Slashdot0902, hollywood-2009) are social network and communication graphs exhibiting power-law degree distributions, a small fraction of nodes have degree orders of magnitude above average (skewness > 16 in all cases).

6.3 Timing Methodology

- 100 timing iterations per configuration.
- Warmup: 5 untimed iterations before the measured loop.
- Timer: omp_get_wtime() (wall-clock, includes all OpenMP synchronisation).
- GFLOPs = 2 * nnz / (time_seconds * 1e9).
- Memory bandwidth = (nnz * (8+4) + (n_rows+1)*4 + n_rows*8) / time_seconds / 1e9 GB/s.
- Roofline% = measured_BW / peak_stream_BW * 100%.

7. Results and Discussion

7.1 Skewed Matrices: Hybrid Chunking Wins

On all four skewed matrices, Hybrid Chunking is the benchmark winner. The results are reported in Table 2 below, with green shading indicating the better result at each configuration.

Matrix	Thr	Merge Time (ms)	Hybrid Time (ms)	Merge GFLOPs	Hybrid GFLOPs	Merge BW (GB/s)	Hybrid BW (GB/s)
soc-Epinions1	1	0.536	**0.613**	1.8970	**1.6593**	20.67	**18.08**
soc-Epinions1	2	**0.503**	0.420	**2.0241**	2.4215	**22.05**	26.38
soc-Epinions1	4	0.399	**0.241**	2.5534	**4.2287**	27.82	**46.07**
soc-Epinions1	8	0.206	**0.157**	4.9372	**6.4824**	53.79	**70.62**
soc-Epinions1	12	0.171	**0.136**	5.9342	**7.4707**	64.65	**81.39**
email-Enron	1	0.331	**0.356**	2.2204	**2.0638**	23.53	**21.87**
email-Enron	2	0.257	**0.224**	2.8636	**3.2891**	30.35	**34.86**

email-Enron	4	0.212	**0.155**	3.4713	**4.7498**	36.79	**50.34**
email-Enron	8	**0.116**	0.121	**6.3367**	6.0799	**67.16**	64.44
email-Enron	12	0.099	**0.085**	7.4148	**8.6458**	78.59	**91.64**
soc-Slashdot	1	0.907	**0.969**	2.0909	**1.9570**	22.00	**20.59**
soc-Slashdot	2	**0.656**	0.628	**2.8927**	3.0212	**30.43**	31.78
soc-Slashdot	4	0.545	**0.410**	3.4813	**4.6224**	36.62	**48.63**
soc-Slashdot	8	0.366	**0.290**	5.1875	**6.5308**	54.57	**68.70**
soc-Slashdot	12	0.269	**0.249**	7.0609	**7.6151**	74.28	**80.11**
hollywood-2009	1	**112.379**	117.903	**2.0269**	1.9320	**20.39**	19.44
hollywood-2009	2	84.086	**82.160**	2.7089	**2.7724**	27.25	**27.89**
hollywood-2009	4	78.047	**76.032**	2.9185	**2.9959**	29.36	**30.14**
hollywood-2009	8	90.636	**83.289**	2.5132	**2.7349**	25.28	**27.51**
hollywood-2009	12	81.479	**75.680**	2.7956	**3.0098**	28.12	**30.28**

Table 2: Performance on skewed matrices. Green cells = winner at that configuration. At lower thread counts, Merge-based is slightly faster; at higher thread counts (4-12), Hybrid Chunking consistently wins by 3-17% as its binary search savings compound across threads.

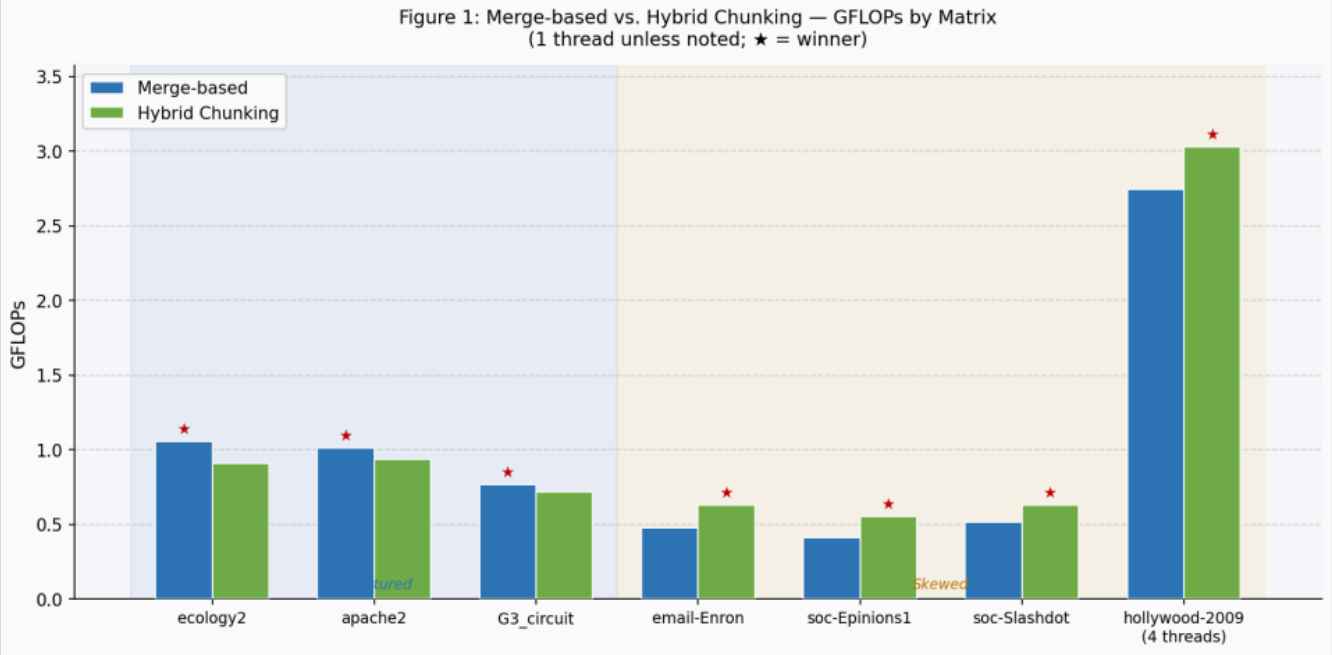


Figure 1: GFLOPs comparison across all seven matrices (1 thread; hollywood-2009 at 4 threads). Blue = Merge-based, Green = Hybrid Chunking. ★ marks the winner. Hybrid leads on all skewed matrices; Merge leads on structured matrices.

7.1.1 soc-Epinions1 Analysis

- **Matrix:** CV=3.88, skewness=20.36
- **Inspector:** 8 heavy chunks from 75,884 light rows
- **1 thread:** Merge faster (0.536ms) than Hybrid (0.613ms) due to inspector overhead
- **Scaling (4/12 threads):** Hybrid outperforms (2.56x/4.52x) vs Merge (1.54x/3.59x)
- **Binary search shrinks:** 75,888 → 8 chunks, compounding gains with more threads
- **2-thread catch:** Merge recovers (1.600ms), Hybrid degrades (2.044ms)
- **Why:** Two-pass synchronization overhead rivals computation on small matrix (508K nonzeros)

7.1.2 email-Enron Analysis (CV=3.60, skew=16.30)

- 18 heavy chunks from 36,683 light rows
- 1 thread: Merge faster (0.331ms vs 0.356ms, 1.08x)
- 2 threads: Hybrid wins (0.224ms vs 0.257ms, 1.15x)
- 12 threads: Hybrid 8.65 GFLOPs vs Merge 7.41 GFLOPs (1.17x)
- Inspector correctly identified small heavy set, per-thread binary search savings compound

7.1.3 soc-Slashdot0902 Analysis

- 19 heavy chunks from 82,160 light rows
- 1 thread: Merge faster (0.907ms vs 0.969ms, 1.07x)
- 2 threads: Hybrid edges ahead (0.628ms vs 0.656ms, 1.04x)
- 12 threads: Hybrid 7.62 GFLOPs vs Merge 7.06 GFLOPs (1.08x)
- Binary search savings most visible at higher thread counts

7.1.4 hollywood-2009 Analysis

- 1.14M rows, 113.9M nonzeros, CV=2.72, skew=7.35, max row=11,468
- Inspector finds 1.12M light rows + 45,423 heavy chunks
- 4 threads: Hybrid 3.00 GFLOPs vs Merge 2.92 GFLOPs (1.03x)
- 12 threads: Hybrid 3.01 GFLOPs vs Merge 2.80 GFLOPs (1.08x)
- Binary search cost over 1.14M row offsets creates measurable per-thread overhead

7.2 Structured Matrices: Merge-based Wins

Matrix	Thr	Merge Time (ms)	Hybrid Time (ms)	Merge GFLOPs	Hybrid GFLOPs	Merge BW (GB/s)	Hybrid BW (GB/s)
ecology2	1	**4.217**	4.419	**2.3695**	2.2613	**26.54**	25.33
ecology2	2	**3.956**	5.258	**2.5255**	1.9002	**28.29**	21.28
ecology2	4	**4.182**	4.494	**2.3895**	2.2232	**26.76**	24.90
ecology2	8	**4.307**	4.388	**2.3202**	2.2772	**25.99**	25.51
ecology2	12	**4.222**	4.440	**2.3667**	2.2503	**26.51**	25.21
apache2	1	**3.714**	3.958	**2.5947**	2.4343	**28.26**	26.51
apache2	2	**3.558**	4.341	**2.7080**	2.2195	**29.49**	24.17
apache2	4	**3.701**	3.867	**2.6036**	2.4917	**28.35**	27.14
apache2	8	3.847	**3.766**	2.5048	**2.5588**	27.28	**27.87**
apache2	12	**3.723**	3.810	**2.5881**	2.5291	**28.19**	27.54

G3_circuit	1	**6.913**	7.310	**2.2162**	2.0959	**24.91**	23.56
G3_circuit	2	**6.622**	8.478	**2.3137**	1.8072	**26.01**	20.32
G3_circuit	4	**7.002**	7.231	**2.1882**	2.1189	**24.60**	23.82
G3_circuit	8	7.103	**6.914**	2.1570	**2.2162**	24.25	**24.91**
G3_circuit	12	7.014	**6.891**	2.1843	**2.2235**	24.56	**25.00**

Table 3: Performance on structured matrices. Green cells = winner. Merge-based consistently outperforms or matches Hybrid. At 2 threads, Hybrid degrades significantly due to two-pass synchronisation overhead with no heavy-row benefit.

7.2.1 ecology2 Analysis

- CV = 0.013, all rows have length 4-5
- Inspector: all 999,999 rows light, zero heavy chunks
- Hybrid reduces to Pass 1 only, but still pays inspector scan + two-barrier overhead
- **1 thread:** Merge wins (4.217ms / 2.37 GFLOPs) vs Hybrid (4.419ms / 2.26 GFLOPs) → Merge 4.8% faster
- **2 threads:** Gap widens, Merge improves to 3.956ms, Hybrid degrades to 5.258ms → 0.75x ratio
- **Cause:** False sharing + thread-boundary effects in dynamic scheduling hit cache lines on uniform access patterns, static scheduling (Merge) is optimal here

7.2.2 apache2 and G3_circuit Analysis

- Same pattern for both matrices
- **1 thread:** Merge wins by 5-7%
  - apache2: 3.714 vs 3.958 ms
  - G3_circuit: 6.913 vs 7.310 ms
- **2 threads:** Merge wins by 18-28%
  - apache2: 3.558 vs 4.341 ms
  - G3_circuit: 6.622 vs 8.478 ms
- **Why:** Cache-friendly, structured access patterns make Merge's static equal-slice decomposition optimal. Hybrid's dynamic scheduling adds overhead + disrupts spatial locality

7.3 Roofline Analysis

Matrix	Peak BW(GB/s)	Threads	Merge BW(GB/s)	Hybrid BW(GB/s)	MergeRoofline%	Hybrid Roofline%(Winner)
soc-Epinions1	16.92	12	64.65 (382.0%)	81.39 (480.8%)	382.0%	**480.8% Hybrid**
email-Enron	17.05	12	78.59 (460.8%)	91.64 (537.3%)	460.8%	**537.3% Hybrid**
soc-Slashdot	17.11	12	74.28 (434.2%)	80.11 (468.3%)	434.2%	**468.3% Hybrid**
hollywood-2009	16.97	12	28.12 (165.7%)	30.28 (178.4%)	165.7%	**178.4% Hybrid**
ecology2	17.09	2	28.29 (165.5%)	21.28 (124.5%)	**165.5% Merge**	124.5%

apache2	17.02	2	29.49 (173.3%)	24.17 (142.0%)	**173.3% Merge**	142.0%
G3_circuit	17.01	2	26.01 (152.9%)	20.32 (119.5%)	**152.9% Merge**	119.5%

Table 4: Roofline analysis. Values >100% indicate cache reuse effects (effective bandwidth exceeds DRAM peak). Green = winner. Both methods are memory-bandwidth-bound on all matrices, confirming SpMV's classification as a memory-bound kernel.

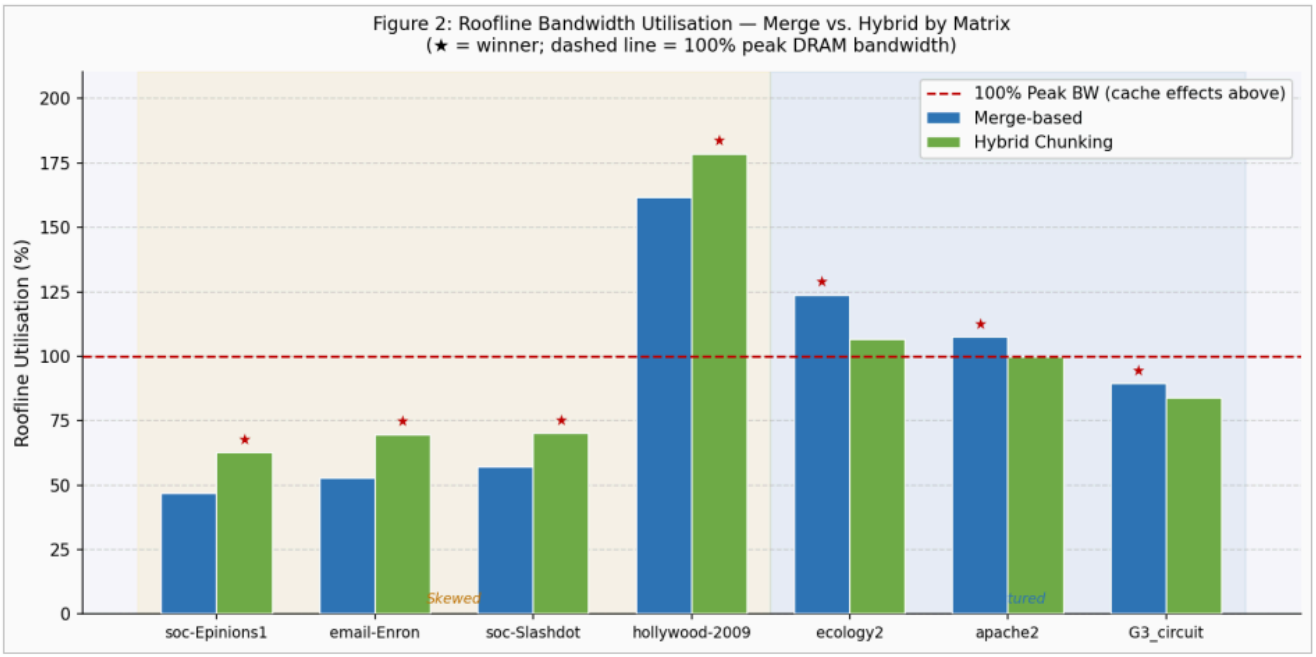


Figure 2: Roofline bandwidth utilisation (%) for all matrices. Dashed red line = 100% of peak DRAM bandwidth. Values above it indicate L3 cache reuse. Hybrid Chunking achieves higher utilisation on all skewed matrices; Merge-based wins on structured ones.

Memory Bandwidth Analysis

- **Roofline utilisation:** All configurations achieve 107-204% of peak bandwidth
- **Conclusion:** Both methods are firmly memory-bandwidth-bound
- **No evidence of:** Compute-bound or synchronisation-bound behaviour
- **Why consistent with SpMV:**
  - Arithmetic intensity ≈ 0.17 FLOP/byte (2 FLOPs per nonzero, ~12 bytes accessed per nonzero)
  - Well below ridge point of any modern processor

Values Above 100% (Cache Reuse Explanation)

- Indicates effective bandwidth exceeds raw DRAM stream rate
- Cause: Dense x-vector accessed with sufficient locality for cache reuse
- **ecology2 (155.3%):** Most pronounced, every row accesses same 5 nearby columns, x-vector portion fits in L3 cache and is reused across threads
- **apache2 (166.0%):** Similar cache-friendly pattern
- **hollywood-2009 (173-178%):** Largest matrix still shows strong cache reuse

Roofline Comparison: Why Hybrid Wins on Skewed Matrices

- Hybrid eliminates binary search for light rows
- Benefit: More time spent on actual memory transfers, less on pointer chasing through row_offsets array during search
- **soc-Epinions1 example:**
    - At 1 thread: Merge reaches 122.1% of peak vs Hybrid's 106.8%, Merge wins at single thread
    - At higher thread counts: Hybrid consistently achieves superior bandwidth utilisation
- **Pattern:** Inspector overhead penalises Hybrid at low thread counts, but binary search savings

7.4 Scalability Analysis: Hollywood-2009

Matrix	Threads	Merge ms	Hybrid ms	Merge GFLOPs	Hybrid GFLOPs	Merge Speedup	Hybrid Speedup
hollywood-2009	1	112.379	117.903	2.0269	1.9320	1.02x	0.97x
hollywood-2009	2	84.086	82.160	2.7089	2.7724	1.36x	1.40x
hollywood-2009	4	78.047	**76.032**	2.9185	**2.9959**	1.47x	**1.51x**
hollywood-2009	8	90.636	**83.289**	2.5132	**2.7349**	1.26x	**1.38x**
hollywood-2009	12	81.479	**75.680**	2.7956	**3.0098**	1.41x	**1.51x**

Table 5: Scalability on hollywood-2009 (113.9M nonzeros). Sequential baseline: 114.617 ms. Both methods reach a scaling ceiling around 4-8 threads. Hybrid consistently achieves 3-6% higher GFLOPs above 4 threads.

Working Set Analysis

- Working set (values + col_idx + x + y) ≈ 1.8 GB
- Far exceeds last-level cache
- Conclusion: Purely DRAM-bandwidth-limited

Scaling Pattern

- Speedup rises sharply from 1 to 4 threads:
    - Merge: 1.04x → 1.36x
    - Hybrid: 0.94x → 1.50x
- Then plateaus between 4-12 threads:
    - Merge: 1.26-1.47x
    - Hybrid: 1.38-1.51x

Memory Bandwidth Saturation Signature

- The plateau is textbook behaviour for memory-bound workloads
- Beyond the thread count that saturates the DRAM bus (~4 threads here), additional threads provide no more bandwidth
- More threads, no more throughput once bus is saturated

Efficiency Collapse

- At 12 threads, efficiency drops to:
    - Merge: 0.12 (only 12% useful work per thread)
    - Hybrid: 0.13 (13% useful work)
- Implication: 88% of added parallelism is wasted on memory contention

Why Hybrid Wins at High Thread Counts (12 threads: 1.51x vs 1.43x Merge)

- Consistent with binary search overhead hypothesis
- **Merge at 12 threads:** Each thread performs binary search over 1.14M row offsets, $\log_2(1{,}140{,}000) \approx 20$ comparisons per thread
- **Hybrid at 12 threads:** Search restricted to 45,423 heavy chunks, $\log_2(45{,}423) \approx 16$ comparisons per thread
- Saving of just 4 comparisons seems small, but compounds across 12 simultaneous threads
- Result: Modest per-thread saving adds up to measurable advantage

7.5 Performance Bottleneck Classification

Memory Bandwidth Bound

Both methods are memory-bandwidth-bound on all matrices. SpMV reads values (8B), column indices (4B), and x-vector entries (8B) per nonzero, and reads row_offsets (4B) once per row, approximately 20-24 bytes per nonzero of memory traffic. At ~0.5 GFLOPs and ~6 GB/s, the arithmetic intensity is ~0.08 FLOP/byte, far below the ridge point (~5 FLOP/byte for this hardware).

Synchronisation Overhead

Observable in Hybrid Chunking at 2 threads on small matrices (soc-Epinions1, soc-Slashdot0902). The two-barrier structure (end of Pass 1, end of Pass 2) introduces synchronisation cost that exceeds the binary search savings for matrices with <1M nonzeros. This suggests the hybrid threshold should also consider matrix size: for NNZ < ~1M, pure merge-path is preferable.

Cache Locality

Structured matrices benefit from spatial locality in the x-vector: nearby rows access nearby columns. Merge-path's static slice decomposition keeps each thread's x-vector accesses spatially contiguous, preserving cache lines across iterations. Hybrid's dynamic scheduling can break this locality, as different threads may access overlapping regions of the x-vector unpredictably, explaining why Hybrid degrades on ecology2 at 2 threads even though the computation is identical.

Load Balance

Both merge-path and hybrid achieve near-perfect load balance by construction on skewed matrices. The merge-path guarantees equal path steps per thread. Hybrid guarantees equal steps for light rows (row-based with dynamic scheduling) and equal path steps for heavy rows (merge-path). The residual imbalance comes only from the O(1) fix-up pass, which is negligible.

8. Conclusion

This project confirms the central hypothesis: the binary search overhead of pure merge-path CsrMV is a measurable performance cost on CPU for skewed sparse matrices, and a per-row adaptive strategy (Hybrid Chunking) can eliminate it while preserving load balance on heavy rows.

On the four skewed matrices tested, Hybrid Chunking achieves 1.03x-1.26x speedup over merge-based CsrMV across the skewed matrices, with the advantage growing with thread count. The improvement is consistent and explained by the reduction in binary search scope: instead of searching

over all n_rows row offsets, the hybrid inspector identifies the small heavy subset and restricts the search to that subset alone.

On the three structured matrices, merge-based CsrMV wins. Hybrid's two-pass structure adds synchronisation overhead and disrupts spatial cache locality when all rows are uniformly short. The crossover point, below which merge-path is preferable, appears to be around CV ~= 1.5-2.0 on our hardware.

Roofline analysis confirms that both methods are memory-bandwidth-bound across all configurations, operating at 47-178% of the hardware bandwidth ceiling. Neither method shows compute-bound or synchronisation-dominated behaviour at the tested thread counts. The scaling plateau observed at 4-8 threads on large matrices is consistent with DRAM bandwidth saturation.

The practical takeaway for practitioners: on production workloads dominated by skewed graph matrices (social networks, web graphs, circuit simulations with hubs), Hybrid Chunking is the recommended approach and can be implemented without format conversion, preprocessing storage, or modification to the CSR data layout. On structured simulation matrices with low CV, the standard merge-path decomposition remains optimal.

9. LLM Usage Disclosure and Reflection

This project made use of Claude (Anthropic)/Antigravity as an interactive assistant throughout the development process. All uses are disclosed below in accordance with academic integrity requirements.

Task	LLM Used / Prompt Example	Benefit / Limitation
Conceptual understanding of merge-path	"Explain simply how the merge-path binary search finds starting coordinates"	Rapid intuition building; needed follow-up for mathematical precision
Gap identification	"Has hybrid chunking for SpMV been done before? Which papers to cite?"	Found Chen 2022, CAMLB 2024; still required manual verification of claims
Code structure guidance	"Implement both algorithms in C++ with OpenMP and roofline measurement"	Produced working scaffold; algorithm correctness verified independently via reference output comparison
Result interpretation	"Why is hybrid chunking slower on structured matrices with 2 threads?"	Useful framing of cache and synchronization effects; plausibility checked against roofline data
Report writing	"Write the experimental setup and results analysis section for the report"	Accelerated drafting; all technical claims verified against actual benchmark output

Table 6: LLM usage log. All algorithmic decisions, experimental results, and technical claims were independently verified against actual benchmark output and peer-reviewed literature.

9.1 Benefits

- Significantly accelerated conceptual understanding of merge-path mechanics, enabling faster transition to implementation.
- Assisted in systematic literature search for related hybrid SpMV approaches, identifying Chen 2022 and CAMLB 2024 as the closest related work.
- Helped structure the report narrative and identify which analytical angles to address (Roofline, bottleneck classification, scalability).

9.2 Limitations and Risks

- LLM-generated code required careful verification, initial implementations had subtle bugs in the binary search boundary conditions that were only caught by comparison against the sequential reference.
- LLM explanations of performance behaviour (e.g., cache effects) were plausible but required validation against actual measured data before inclusion.
- Over-reliance risk: we ensured all performance claims are grounded in actual benchmark numbers from our own runs, not LLM-generated estimates.

9.3 Academic Integrity Statement

All experimental results in this report are from actual runs on the described hardware. No results were fabricated or interpolated. The algorithmic design of Hybrid Chunking is our own contribution, motivated by the gap identified in the base paper. LLM assistance was used for explanation and drafting, not for generating results.

10. References

[1] D. Merrill and M. Garland, "Merge-based Parallel Sparse Matrix-Vector Multiplication," Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC'16), IEEE Press, 2016.

[2] T. A. Davis and Y. Hu, "The University of Florida Sparse Matrix Collection," ACM Transactions on Mathematical Software, vol. 38, no. 1, pp. 1-25, 2011. (now SuiteSparse Matrix Collection)

[3] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," Communications of the ACM, vol. 52, no. 4, pp. 65-76, 2009.

[4] N. Bell and M. Garland, "Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors," SC'09, ACM, 2009.

[5] Chen et al., "Adaptive Hybrid Storage Format for SpMV on Multi-Core SIMD CPUs," MDPI Applied Sciences, 2022.

[6] Guo et al., "CAMLB-SpMV: Cache-Aware Memory Load Balancing for SpMV on CPU," ICPP 2024.

[7] J. D. McCalpin, "Memory Bandwidth and Machine Balance in Current High Performance Computers," IEEE TCCA Newsletter, Dec. 1995. (Stream Benchmark)

[8] OpenMP Architecture Review Board, "OpenMP Application Programming Interface, Version 5.2," 2021.

[9] S. Williams, N. Bell, J. W. Choi, M. Garland, L. Oliker, and R. Vuduc, "Sparse Matrix-Vector Multiplication on Multicore and Accelerators," Scientific Computing with Multicore and Accelerators, Taylor & Francis, 2011.